

Jogo de Corrida Distribuído

Distributed Racing Game

**Bernardo Temponi¹, Gabriel Schlegel¹, Júlio César¹, Lucas Cabral¹,
Pedro Gaioso¹, Priscila Goulart¹, Rafael Vilefort¹**

¹Instituto de Informática – Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Caixa Postal 1.686. – 30535-901 – Cidade – BH – Brasil

Abstract. Introduction: *This paper presents the development of a distributed multiplayer racing game focused on high availability and state consistency. The architecture employs a server cluster where nodes coordinate using the Bully leader election algorithm and synchronize events via Lamport logical clocks. Communication is handled through a custom RPC protocol implemented over TCP. The system ensures resilience by allowing automatic client redirection and leadership recovery in the event of node failures, maintaining a synchronized game state across the network.*

Keywords *Distributed Systems, Leader Election, Bully Algorithm, Lamport Clocks, Multiplayer Games.*

Resumo. *Este artigo apresenta o desenvolvimento de um jogo de corrida multiplayer distribuído focado em alta disponibilidade e consistência de estado. A arquitetura emprega um cluster de servidores onde os nós se coordenam utilizando o algoritmo de eleição de líder Bully e sincronizam eventos por meio de relógios lógicos de Lamport. A comunicação é gerenciada através de um protocolo RPC próprio implementado sobre TCP. O sistema garante resiliência ao permitir o redirecionamento automático de clientes e a recuperação da liderança em caso de falhas de nós, mantendo o estado do jogo sincronizado em toda a rede.*

Palavras-Chave *Sistemas Distribuídos, Eleição de Líder, Algoritmo Bully, Relógios de Lamport, Jogos Multiplayer.*

1. Explicação do Problema Escolhido

O desenvolvimento de jogos *multiplayer* em tempo real apresenta um dos desafios mais complexos no campo dos Sistemas Distribuídos: a manutenção de um estado global consistente e fluido sob condições de rede imprevisíveis. O problema escolhido para este trabalho é a implementação de um **Jogo de Corrida Distribuído**, onde múltiplos jogadores competem simultaneamente em um ambiente compartilhado.

Neste cenário, o sistema deve gerenciar dois problemas fundamentais:

- **Disponibilidade e Tolerância a Falhas:** Em arquiteturas cliente-servidor tradicionais, a queda do servidor central interrompe a partida imediatamente. O problema abordado é como manter a partida ativa mesmo que o nó líder sofra uma falha crítica, utilizando um *cluster* de servidores com eleição dinâmica.
- **Consistência e Ordenação de Eventos:** Como as mensagens de *input* chegam em momentos diferentes, surge o problema de como ordenar esses eventos de forma idêntica em todos os nós. Sem sincronização lógica, a integridade da simulação física seria quebrada[.

2. Detalhamento da Arquitetura do Sistema

A arquitetura do sistema é baseada em um modelo de Cluster Cooperativo com Replicação de Estado, composta por três entidades principais: o Nó Líder, os Nós Seguidores e os Clientes. A comunicação entre essas entidades é realizada através de um protocolo RPC (Remote Procedure Call) customizado sobre o protocolo de transporte TCP.

2.1. Papéis das Entidades

- **Nó Líder (Leader):** É a autoridade central do cluster no momento. Sua função é processar a lógica da física do jogo, consolidar os comandos recebidos dos clientes e difundir o estado atualizado para todos os seguidores e clientes ativos.
- **Nó Seguidor (Follower):** Atua como um nó de espera ativa (hot standby). Ele mantém uma cópia espelhada do estado do jogo e monitora o líder através de mensagens de *healthcheck*. Caso detecte a ausência do líder, está apto a iniciar um processo de eleição.
- **Cliente (Pygame):** Atua como a interface do usuário. Captura os eventos do teclado, envia para o líder e renderiza o estado do jogo recebido. O cliente possui inteligência de rede para se reconectar automaticamente caso o líder mude.

2.2. Fluxo de Mensagens e Conteúdo

A troca de mensagens ocorre de forma contínua para garantir a sincronia do jogo. As principais interações são:

1. **Input do Jogador (Cliente → Líder):** O cliente envia uma mensagem contendo o ID do jogador, o vetor de movimento atual e o *timestamp* de Lamport local.
2. **Replicação de Estado (Líder → Seguidores/Clientes):** O líder envia periodicamente (aproximadamente 15 vezes por segundo) um pacote contendo a posição (x, y) de todos os carros, velocidades, colisões detectadas e o *timestamp* lógico atualizado.
3. **Monitoramento (Seguidor ↔ Líder):** Troca de mensagens curtas de *Ping/Pong* para verificar a latência e a vivacidade do nó central.
4. **Redirecionamento (Nó → Cliente):** Se um cliente tentar conectar-se a um nó que não é o líder, este responde com uma mensagem de *Redirect* contendo o endereço IP e a porta do líder atual.

Para garantir a confiabilidade, cada mensagem RPC possui um cabeçalho fixo que identifica o tipo da operação e o tamanho do conteúdo, permitindo que o receptor processe os bytes corretamente no fluxo TCP.

3. Detalhamento da Implementação dos Requisitos

3.1. Protocolo RPC Customizado sobre TCP

Implementou-se um protocolo de **Chamada de Procedimento Remoto (RPC)** próprio.

- **Transporte:** Utiliza *sockets* TCP para garantir entrega ordenada.
- **Serialização:** Cada pacote possui um cabeçalho que define o tipo da operação (ex: INPUT, STATE_UPDATE, ELECTION) e o tamanho do *payload*.

3.2. Eleição de Líder (Algoritmo Bully)

O sistema utiliza o **Algoritmo Bully** para garantir um coordenador ativo.

- **Deteção:** Se o líder não responder em 0,5 segundos, o processo de eleição é disparado.
- **Fluxo:** O nó envia mensagens de eleição para IDs superiores. Se não houver resposta, autoproclama-se líder.

3.3. Sincronização por Relógios Lógicos de Lamport

Implementou-se o **Relógio Lógico de Lamport** para ordenação causal.

- **Timestamping:** Cada mensagem trafegada incrementa o contador local seguindo a regra $L_{local} = \max(L_{local}, L_{mensagem}) + 1$.

4. Desafios e Problemas Encontrados

O desenvolvimento de um jogo de corrida com arquitetura distribuída apresentou desafios técnicos significativos, principalmente no que tange à coordenação de processos e à tolerância a falhas em tempo real.

- **Ajuste de Timeouts e Falsos Positivos:** Um dos maiores desafios foi calibrar o tempo de detecção de falha do líder. Definir um *timeout* muito curto (inferior a 0,3s) causava eleições desnecessárias devido a oscilações momentâneas na latência da rede local. O valor estável foi encontrado em 0,5s, equilibrando rapidez na recuperação e estabilidade do cluster.
- **Gerenciamento de Concorrência (Python GIL):** Como o Python utiliza o *Global Interpreter Lock* (GIL), a execução de múltiplas *threads* para gerenciar a interface gráfica do Pygame, a lógica da física e a escuta de diversos *sockets* TCP exigiu um design cuidadoso[cite: 1]. Foi necessário garantir que o processamento intenso de mensagens de rede não bloqueasse a renderização do jogo, o que causaria "travamentos" visuais para o usuário[cite: 1].
- **Consistência no Redirecionamento:** Implementar a transição transparente do cliente entre diferentes servidores foi complexo[cite: 1]. O desafio consistiu em garantir que, ao receber um comando de *redirect*, o cliente pudesse se conectar ao novo líder e sincronizar seu estado local através do relógio de Lamport sem que houvesse saltos bruscos ou perda de pacotes de *input* durante a troca[cite: 1].
- **Tratamento de Nós em Cooldown:** Durante os testes, percebeu-se que nós que falhavam e retornavam rapidamente podiam causar instabilidade no Algoritmo Bully[cite: 1]. A solução foi implementar um período de *cooldown* de 3 segundos antes que um nó pudesse tentar restabelecer contato com o cluster, evitando ciclos infinitos de eleição[cite: 1].

5. Condução do Trabalho

Dada a complexidade do sistema distribuído e da interface gráfica, o grupo dividiu-se em frentes de trabalho coordenadas para garantir a integração dos requisitos obrigatórios.

- **Gabriel Schlege (Líder de Backend):** Responsável pela arquitetura do cluster e implementação do protocolo RPC customizado sobre sockets TCP.
- **Bernardo Temponi (Especialista em Sistemas Distribuídos):** Implementou a lógica do Algoritmo Bully para a eleição de líder e a detecção de falhas entre os nós.
- **Pedro Gaioso (Desenvolvedor de Consistência):** Responsável pela implementação e integração dos Relógios Lógicos de Lamport em todas as mensagens do sistema.
- **Júlio César (Desenvolvedor de Jogo/Física):** Implementou a lógica de movimentação, colisão e aceleração utilizando a biblioteca Pygame.

- **Rafael Vilefort (Engenheiro de Redes):** Responsável pela lógica de redirecionamento automático do cliente e tratamento de concorrência/threads no servidor.
- **Priscila Goulart (Analista de Testes e QA):** Criou os scripts de teste de carga e simulação de falhas (derrubando o líder) para validar a resiliência do cluster.
- **Lucas Cabral (Documentação e DevOps):** Responsável pela elaboração do relatório técnico seguindo o template da SBC e configuração do ambiente de execução e README.

6. Referências

- Coulouris, G., Dollimore, J., Kindberg, T., e Blair, G. (2013).** *Sistemas Distribuídos: Conceitos e Projeto*. Bookman Editora, 5ª edição.
- Garcia-Molina, H. (1982).** Elections in a Distributed Computing System. *IEEE Transactions on Computers*, C-31(1):48–59. (Referência clássica do Algoritmo Bully).
- Lamport, L. (1978).** Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565. (Referência fundamental para Relógios Lógicos).
- Tanenbaum, A. S. e Van Steen, M. (2007).** *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2ª edição.